Cordova Public College

Backend Technical Documentation

A complete, plain-language explanation of every file, every decision, and every layer of the Rootine API.

Members

Capstone Project · 2026

Chapter 1 — What is Rootine?

Imagine you want to drink 8 glasses of water every day, go to the gym 3 times a week, and read for 30 minutes each morning. You try to remember all of this in your head — and you forget. You try writing it on paper — the paper gets lost.

Rootine solves this. It is a habit-tracking app where users can list their habits, mark them done every day, and watch their progress grow over time. Think of it like a personal coach who keeps track of your routines — and reminds you when you forget.

What this document covers

This document explains the **backend** — the behind-the-scenes engine that powers Rootine. The backend is like the kitchen of a restaurant. The customer (the app on your phone) orders food, but all the cooking, storing, and preparing happens in the kitchen — out of sight, but absolutely essential.

The backend handles:

- **Accounts** — signing up, logging in, verifying your email, resetting passwords
- **Tasks/Habits** — creating, editing, deleting, archiving, and reordering your habits
- **Completions** — recording when you finish a habit each day
- **Streaks** — calculating how many days in a row you've kept a habit
- **Reminders** — sending you emails so you don't forget your habits
- **Settings** — letting you update your name, email, phone, password, and timezone

Chapter 2 — How the Backend is Organized

The backend follows a **layered architecture**. Think of it like a company with different departments — each department has one job and doesn't interfere with the others.

Layer	Department Analogy	Job
Routes	Reception desk	Decides which request goes where
Controllers	Call center agents	Receives the request, sends it to the right service
Services	The decision-makers	Business logic — figures out what to do
Repositories	The file clerks	All database read/write operations
DB Schema	The filing cabinet blueprint	Defines how data is stored
Middlewares	Security guards	Check identity and validate inputs before anything runs
Utils	Toolbox	Shared helper tools used by many layers

Why organize it this way? Because if something breaks, you know exactly where to look. If a reminder isn't being sent — check the services/reminder folder. If the database isn't saving data correctly — check the repositories. Each part is isolated **so that** one problem doesn't affect everything else.

Chapter 3 — File-by-File Explanation

server.ts

src/server.ts

■ What is it?

This is the **entry point** — the very first file that runs when you start the Rootine backend. It imports the configured app and tells it to start listening for incoming requests.

■ Why does it exist?

Because something has to turn the engine on. If you build a car but never turn the key, it goes nowhere. **server.ts is the ignition.** Without it, the entire backend sits idle — no requests are accepted, no emails are sent, nothing works.

■ Purpose

Start the HTTP server on the configured port and launch the background reminder worker so email reminders begin running immediately.

■ When does it run?

Every single time you run npm run dev or node dist/server.mjs. It runs once at startup and stays alive until the server is shut down.

■ Real-life example

Think of it like opening a store. server.ts is the manager who arrives first, unlocks the door (starts the server), and turns on the lights (starts the reminder worker) — **so that** everything is ready before the first customer walks in.

app.ts

src/app.ts

■ What is it?

This file sets up the Express application — all the security layers, parsing tools, and route connections. Think of it as the **building layout** of the entire backend.

■ Why does it exist?

Because a server without security is like a bank with no locks. **If we skipped this file**, anyone could send unlimited requests, inject malicious data, or crash the server with a single huge message. Every protection we add here shields all routes at once.

■ Purpose

Configure the Express app with: rate limiting, CORS, security headers (Helmet), body size limit, cookie parsing, and all route registrations — **in order for** every request to pass through the right checks before reaching any controller.

■ When does it run?

Once at startup (when server.ts imports app.ts). After that, the configuration stays loaded for the lifetime of the server.

■ Real-life example

Imagine airport security. Before you reach your gate (your controller), you go through: check-in (rate limiter), passport control (auth middleware), bag X-ray (body parser), customs (CORS). **app.ts is the airport layout that defines all those checkpoints.**

What if we removed rate limiting? Then a hacker could try 10,000 passwords per minute on the login endpoint. With rate limiting: only 10 attempts per 15 minutes per IP — then the door locks automatically.

env.ts

src/config/env.ts

■ What is it?

Loads all environment variables (from the .env file) into one clean object called ENV. Every other file gets its settings from this single source.

■ Why does it exist?

Because hardcoding secrets (like passwords or database URLs) directly in source code is dangerous — if someone sees your code, they see your secrets. **env.ts keeps secrets separate from code**, and it throws an error immediately if a required secret is missing.

■ Purpose

Be the single source of truth for all configuration values — **so that** every file reads settings from one place and the app refuses to start if critical settings like JWT_SECRET or DATABASE_URL are missing.

■ When does it run?

At server startup, before anything else runs. **If a required variable is missing, the app crashes immediately** with a clear message — better to know at startup than to crash mysteriously 10 minutes into serving users.

■ Real-life example

Think of it like a pilot's pre-flight checklist. Before the plane takes off, the pilot confirms fuel, engines, radio. **If the fuel gauge is missing, the plane doesn't take off.** env.ts is that checklist — it checks all critical settings before the server 'takes off'.

db/schema.ts

src/db/schema.ts

■ What is it?

Defines the exact structure of every table in the PostgreSQL database — users, tokens, tasks, and task_completions. This is the **blueprint of the data storage system**.

■ Why does it exist?

Because a database without a clear structure is chaos. **What if** one part of the app saves a user's email as 'email' and another part looks for it as 'e_mail'? Nothing would match. schema.ts enforces one consistent structure **so that** every part of the app speaks the same language.

■ Purpose

Define all database tables, their columns, their types, their rules (required/optional, unique, default values), and their relationships — **in order for** Drizzle ORM to know exactly how to read and write data.

■ When does it run?

Never runs directly at request time — it's a definition file. It's used by Drizzle when generating migrations and by repositories whenever they query the database.

■ Real-life example

Think of it like designing a form. Before printing 1,000 registration forms, you design the layout — Name (required), Email (unique, required), Phone (required). **schema.ts is that form design.** Once set, everyone fills in the same fields in the same format.

Tables defined: users (accounts), tokens (refresh/verify/reset tokens), tasks (habits), task_completions (daily logs). Each table has auto-generated UUIDs as IDs — like automatically assigning a unique student ID to every new enrolment.

db/index.ts

src/db/index.ts

■ What is it?

Creates and exports the single database connection instance (db) using Drizzle ORM and the DATABASE_URL from env.ts.

■ Why does it exist?

Because we need exactly one connection to the database — not one per file, not one per request. **If every file opened its own connection**, we'd exhaust the database's connection limit in seconds. One shared connection pool handles everything efficiently.

■ Purpose

Open a connection to the PostgreSQL database once and share it across all repositories — **so that** the entire backend uses one managed pool of connections.

■ When does it run?

Once at startup. Every repository imports db from this file whenever it needs to query or write data.

■ Real-life example

Like a shared office printer. Instead of everyone buying their own printer, one printer is set up in the office and everyone sends their jobs to it. **db/index.ts is that shared printer — one connection, many users.**

auth-middleware.ts

src/middlewares/auth-middleware.ts

■ What is it?

A security guard that checks every protected request for a valid JWT access token. If the token is missing, expired, or fake — the request is rejected before it reaches any controller.

■ Why does it exist?

Because without this, **anyone** could access anyone's tasks by just calling the API directly. The auth middleware is what separates 'logged in' from 'anonymous'. Every protected route uses it.

■ Purpose

Verify the Authorization: Bearer <token> header on incoming requests, decode the user ID from the token, and attach it to the request — **so that** every controller downstream knows who made the request without asking again.

■ When does it run?

On every request to a protected endpoint (tasks, user settings). It runs before the controller — like a checkpoint before the entrance.

■ Real-life example

Imagine a concert. You bought a ticket (logged in and got a token). At the gate, the staff scans it (auth-middleware). **If it's valid — you enter. If it's expired or fake — you're turned away.** The concert hall (controller) never sees people who didn't pass the gate.

What if the token is missing? Returns 401 Unauthorized instantly — the controller never runs.
What if the token is expired? Same result — 401. The user must refresh their token first.

schema-middleware.ts

src/middlewares/schema-middleware.ts

■ What is it?

A validation guard that checks the shape of incoming request data using Zod schemas. If the data doesn't match the expected format, it returns a 400 error with a specific explanation.

■ Why does it exist?

Because controllers should never receive garbage data. **What if** someone sends a registration request with no email? Without this, the signup code would crash trying to process an empty field. **This middleware catches bad data before it ever reaches business logic.**

■ Purpose

Validate every request body against its Zod schema — **so that** controllers always receive clean, correct, typed data and never need to write their own validation checks.

■ When does it run?

On every route that has validation — which is almost all of them. It runs after the auth check but before the controller.

■ Real-life example

Like a bank teller who checks your deposit form before processing it. If you left the amount blank, they hand it back: 'Please fill in the amount.' **They don't process an incomplete form.** schema-middleware does the same for API requests.

error-middleware.ts

src/middlewares/error-middleware.ts

■ What is it?

The last safety net in the application. If any controller or service throws an unexpected error, this middleware catches it and returns a clean, consistent error response instead of crashing.

■ Why does it exist?

Because without it, any unhandled database crash or code bug would leak raw error messages (with file paths, stack traces) to the client — which is both confusing and a security risk. **This keeps errors contained and controlled.**

■ Purpose

Catch any error that bubbles up from controllers or services and return a structured {status: 'error', message: '...'} response — **so that** the client always gets a predictable response, even when something unexpected goes wrong.

■ When does it run?

Only when an error actually occurs — it's the last middleware in the chain. In development it also returns the full stack trace **so that** developers can debug easily.

■ Real-life example

Like a hospital's emergency department. Most people come in through the front door (normal routes), but if something goes wrong anywhere — an accident, a collapse — the ER catches it. **error-middleware is the ER of the backend.**

async-handler.ts

src/middlewares/async-handler.ts

■ What is it?

A tiny wrapper that makes every async controller function automatically forward its errors to the error middleware. Without this, async errors would cause the server to hang or crash silently.

■ Why does it exist?

Because Express (the framework we use) doesn't automatically catch errors from async functions. **If we didn't wrap our controllers**, a database timeout would cause the request to hang forever — the user would wait indefinitely with no response.

■ Purpose

Wrap every async controller **so that** if a database call or service throws any error, it's automatically caught and passed to the error middleware — without writing try/catch in every single controller.

■ When does it run?

It wraps each controller at startup (when routes are defined). The actual catching happens at runtime, whenever an async error occurs.

■ Real-life example

Like a safety net under a tightrope walker. You don't use it until something goes wrong — but the moment the walker slips, the net catches them before they hit the ground. **async-handler is that net for every controller.**

auth-schema.ts

src/schema/auth-schema.ts

■ What is it?

Defines the validation rules for every authentication request: register, login, forgot-password, reset-password, verify-email, resend-verification, refresh-token, and logout.

■ Why does it exist?

Because user input can never be trusted. **What if** someone tries to register with an empty password, or a fake email like 'abc'? auth-schema.ts catches those cases immediately — before any service or database is involved.

■ Purpose

Define exactly what valid input looks like for each auth endpoint — **so that** the validation middleware has clear rules to enforce and services always receive correct data.

■ When does it run?

When a request arrives at any auth route. The schema-middleware applies these rules before the controller runs.

■ Real-life example

Like a visa application form. The form says: passport number must be exactly 9 characters, date of birth must be a real date, photo must be 2×2 inches. **If you submit it wrong, it's rejected at the counter — not after flying to the embassy.**

Password rule: must have uppercase, lowercase, number, and special character. Why? Because 'password123' is one of the most commonly hacked passwords in the world. Enforcing complexity makes brute-force attacks much harder.

task-schema.ts

src/schema/task-schema.ts

■ What is it?

Defines validation rules for task operations — creating, updating, reordering, completing, and uncompleting tasks. Includes rules for reminder settings and schedule configuration.

■ Why does it exist?

Because tasks have many fields with specific rules. **What if** someone sets a reminder time as '25:99' or a days-of-week array with the value 9 (which isn't a real day)? task-schema.ts blocks these before they reach the database.

■ Purpose

Validate all task-related request bodies — **so that** only logically correct tasks are ever saved to the database.

■ When does it run?

Every time a task is created, edited, completed, or reordered.

■ Real-life example

Like a gym registration form that checks: start date must be in the future, package must be one of Bronze/Silver/Gold, payment amount must match the package. Wrong input? Rejected before processing.

user-settings-schema.ts

src/schema/user-settings-schema.ts

■ What is it?

Validation rules for user settings: updating profile fields, requesting an email change, verifying a new email, and changing a password.

■ Why does it exist?

Because user settings are sensitive. **If** someone sends a new password without the required complexity, or an email in the wrong format, this schema catches it immediately.

■ Purpose

Enforce correct formats for all user-settings endpoints — **so that** profile updates, email changes, and password changes always contain valid, safe data.

■ When does it run?

Every time a user tries to update their profile, change their email, or change their password.

Authentication Services

Sign-Up-Service.ts

src/services/auth/Sign-Up-Service.ts

■ What is it?

Handles the full user registration process: check for duplicate email, hash the password, save the user, then send a verification email.

■ Why does it exist?

Because registration is more than just saving data. **If we just saved the raw password**, a database breach would expose every user's password. Hashing means even we can't see it. **If we didn't check for duplicate emails**, two users could share one account.

■ Purpose

Safely create a new user account — **so that** passwords are never stored as plain text and every new user must verify their email before they can log in.

■ When does it run?

When POST /api/v1/auth/register is called.

■ Real-life example

Like a new employee onboarding. HR (Sign-Up) checks: is this person already employed? No? Then creates their employee profile, assigns a secure ID badge (password hash), and sends them a welcome email with an activation link (verification email).

Then the user receives a verification token in their email. **If they never verify** — they can't log in. This prevents fake accounts made with someone else's email.

Log-In-Service.ts

src/services/auth/Log-In-Service.ts

■ What is it?

Handles login: find the user by email, compare the password hash, check email verification, then issue a JWT access token and a refresh token.

■ Why does it exist?

Because logging in involves multiple security checks — not just 'does this email exist?' **If we skipped the emailVerified check**, unverified accounts could log in. **If we didn't hash-compare properly**, any password would work.

■ Purpose

Securely authenticate a user and issue tokens — **so that** only the real owner of a verified account can access protected endpoints.

■ When does it run?

When POST /api/v1/auth/login is called.

■ Real-life example

Like a bank ATM. You insert your card (email), enter your PIN (password). The ATM compares your PIN to the encrypted one on file — not the raw PIN. **If they match AND your account is active (verified)** — you get access. The access token is like a daily temporary passcode.

Access token lasts 10 minutes — short on purpose. **Refresh token** lasts 7 days. **Why two tokens?** The access token is used constantly (so it must be short for safety). The refresh token is stored safely and used only to get a new access token when it expires — like renewing a day pass without signing in again.

Email-Verification-Service.ts

src/services/auth/Email-Verification-Service.ts

■ What is it?

Generates a secure verification token, stores a hashed version in the database, and emails the raw token to the user. Also handles verifying the token when the user submits it.

■ Why does it exist?

Because we need to confirm the user actually owns the email they registered with. **What if** someone registers with your email? You'd receive the verification email, see it's not yours, and ignore it — so the account stays locked. This protects real owners.

■ Purpose

Prove ownership of an email address — **so that** only real people with access to the inbox can activate an account.

■ When does it run?

When a user registers (auto-sent) or when they click 'resend verification'.

■ Real-life example

Like activating a new SIM card. The telecom sends an OTP to your registered number. **If you receive it, you own the number.** If someone registered your number without your permission — you receive the OTP, they don't, account stays inactive.

The token itself is never stored — only its SHA-256 hash is saved. So even if someone accessed the database, they couldn't use the stored value as a token. **Because** SHA-256 is a one-way function — you can't reverse it.

Forgot-Password-Service.ts

src/services/auth/Forgot-Password-Service.ts

■ What is it?

Handles the forgot-password flow: generate a secure reset token, hash and store it, and email it. Also handles the reset-password step that actually updates the password.

■ Why does it exist?

Because people forget passwords. **If there was no reset flow**, a user who forgot their password would be permanently locked out. But the reset must be secure — **if** anyone could reset any password by just knowing the email, accounts could be hijacked.

■ Purpose

Allow legitimate users to regain account access — **so that** only someone with inbox access to the registered email can reset the password.

■ When does it run?

When POST /api/v1/auth/forgot-password and then POST /api/v1/auth/reset-password are called.

■ Real-life example

Like a building with a locked door. You lost your key (forgot password). You call the landlord (forgot-password endpoint) — they send a one-time code to your registered contact. **You use that code to get a new key.** Someone who doesn't have access to your contact can't get the code.

Refresh-Token-Service.ts

src/services/auth/Refresh-Token-Service.ts

■ What is it?

Handles two things: refreshing an access token using a refresh token, and logging out by revoking a refresh token.

■ Why does it exist?

Because access tokens expire every 10 minutes. **If there was no refresh mechanism**, users would have to log in again every 10 minutes — terrible experience. Refresh tokens allow silent re-authentication in the background.

■ Purpose

Keep users logged in without re-entering their password — **so that** the short-lived access token can be replaced automatically before the user even notices it expired.

■ When does it run?

When POST /api/v1/auth/refresh-token is called (usually automatically by the app). Also when POST /api/v1/auth/logout is called.

■ Real-life example

Like a parking ticket that's valid for 10 minutes, but you have a season pass (refresh token) in your pocket. When the ticket expires, you flash the season pass to get a new 10-minute ticket — **without going back to the booth to buy another one.**

Token rotation: every time a refresh token is used, the old one is consumed and a brand new one is issued. **Then if** a hacker steals an old refresh token, it's already been consumed — it no longer works.

Task Services

task-service.ts

src/services/task/task-service.ts

■ What is it?

Contains all the business logic for managing habits/tasks: create, list (paginated), get one, update, soft-delete, archive, unarchive, reorder, and generate the today-dashboard and per-task statistics.

■ Why does it exist?

Because tasks are the core of Rootine. This file is the brain behind everything task-related. **Without it**, there's no way to create habits, no dashboard, no stats — the app has nothing for the user to do.

■ Purpose

Translate requests into correct database actions and return meaningful results — **so that** the user's habits are managed correctly, today's tasks are filtered by their timezone, and streaks are calculated accurately.

■ When does it run?

Every time a user interacts with their tasks — creating, editing, viewing, completing.

■ Real-life example

Like a task manager at a company. Every employee request ('I need a new laptop', 'Schedule a meeting') goes to the task manager, who decides what to do with it, coordinates the right departments, and comes back with a result.

Dashboard optimization: The today-dashboard fetches today's completions in one fast query (status check), then fetches full history per task (streak calculation). **Because** showing wrong streak numbers would be misleading — users trust their streak counts.

task-completion-service.ts

src/services/task/task-completion-service.ts

■ What is it?

Handles marking a task as complete or uncomplete for a given day. Supports multiple completions per day (e.g. drink 8 glasses of water — each glass is one completion).

■ Why does it exist?

Because completing a task is not binary for all habits. **What if** your habit is '8 glasses of water daily'? You need to record each glass, not just 'done/not done'. This service handles those nuances.

■ Purpose

Record, update, or remove completions accurately — **so that** streaks, progress percentages, and statistics are always based on real data.

■ When does it run?

When POST /api/v1/tasks/:id/complete or DELETE /api/v1/tasks/:id/complete is called.

■ Real-life example

Like a punch card at a coffee shop. Each visit, the staff punches one hole. The 10th punch gets you a free coffee. **Each 'complete' call punches one hole**. The system tracks the total count.

reminder-worker.ts

src/services/reminder/reminder-worker.ts

■ What is it?

A background process that runs every 60 seconds, checks all tasks with reminders enabled, and sends email reminders when it's time — respecting quiet hours, daily limits, and stopping once a task is completed.

■ Why does it exist?

Because users forget their habits. **Without reminders**, Rootine is just a to-do list. With reminders, it's an active coach. **But if** reminders were sent without limits, it would become spam. So there are smart rules: quiet hours, max per day, and auto-stop on completion.

■ Purpose

Proactively remind users about incomplete habits — **so that** they stay on track without feeling overwhelmed by notifications.

■ When does it run?

Automatically, every 60 seconds, in the background while the server is running. Also runs a daily token cleanup once per day.

■ Real-life example

Like a personal assistant who checks your calendar every hour. If you haven't marked a meeting as attended and it's between 7AM–10PM, they send you a nudge. **But if you've already attended, they stop nudging.** And they won't bother you at midnight.

Anti-spam rules: (1) Quiet hours: no reminders from 10PM–7AM. (2) Max daily reminders: configurable, default 4. (3) Interval between reminders: configurable, default 1 hour. (4) Auto-stop: once the task is marked complete today, reminders stop until tomorrow.

user-settings-service.ts

src/services/user/user-settings-service.ts

■ What is it?

Handles all user profile management: updating profile fields, requesting an email change (with verification), confirming a new email with a token, and changing the password (requires current password).

■ Why does it exist?

Because users' lives change — they move cities, change phones, change passwords. **If there was no way to update these**, users would be stuck with whatever they entered at registration forever.

■ Purpose

Allow users to keep their profile accurate and secure — **so that** reminders go to the right email, the timezone is correct for streak calculations, and accounts remain secure.

■ When does it run?

When any /api/v1/users/me endpoint is called.

■ Real-life example

Like updating your employee record at HR. You got married and changed your name, moved to a new city, and want a new work email. HR verifies your identity (current password), sends a confirmation to the new email, and only updates after you confirm.

Email change flow: Step 1 — request change (token sent to NEW email). Step 2 — verify with token. **Why two steps?** Because if we updated the email immediately, someone who knows your password could lock you out of your account by changing your email to theirs.

Repositories — The Database Layer

Repositories are the only files allowed to talk directly to the database. Think of them as **librarians** — if you want a book (data), you ask the librarian. You don't go into the back room yourself. **In order for** the rest of the app to stay clean and testable, only repositories touch the database.

user-repository.ts

src/repositories/user-repository.ts

Find users by email or ID, create accounts, update passwords, set email-verified status, and update timezone.

user-settings-repository.ts

src/repositories/user-settings-repository.ts

Update profile fields (name, phone, city, country), update email (and mark unverified), update password hash, check if an email is taken.

token-repository.ts

src/repositories/token-repository.ts

Create tokens, find by hash, mark consumed or revoked, revoke all active tokens for a user+type, and delete expired tokens.

task-repository.ts

src/repositories/task-repository.ts

Create tasks, find by ID+user, list with pagination, update, soft-delete, archive/unarchive, and batch-reorder positions.

task-completion-repository.ts

src/repositories/task-completion-repository.ts

Find completion by task+date, upsert (create or increment), delete, list by task, list by user+date, list all by user.

Utilities — The Toolbox

mailer.ts

src/utils/mailer.ts

■ What is it?

Configures the email transport (Nodemailer with Gmail SMTP or generic SMTP) and provides a single sendEmail() function used everywhere an email needs to be sent.

■ Why does it exist?

Because emails are sent in 4 different places (verification, password reset, task reminders, email-change confirmation). **If each place set up its own email transport**, any change to SMTP settings would require updating 4 files. Centralizing it means one change, all fixed.

■ Purpose

Provide a single, reusable email-sending function — **so that** all emails share the same sender configuration and switching email providers only requires changing one file.

■ When does it run?

Every time any email needs to be sent. If SMTP is not configured in .env, it prints the email to the console instead (useful for development).

■ Real-life example

Like a company's mail room. Every department (HR, Finance, Support) puts letters in the outbox. The mail room handles the actual sending. **Departments don't need their own postage meters.**

timezone.ts

src/utils/timezone.ts

■ What is it?

Helper functions to get the current date, time, hour, and day-of-week in a specific timezone. Used for streak calculations and reminder scheduling.

■ Why does it exist?

Because users live all around the world. **What if** a user in Manila has a habit that resets at midnight — but the server is in UTC? Without timezone conversion, their 'today' resets at 8AM Manila time (UTC+8 midnight), which is completely wrong.

■ Purpose

Provide accurate local time for each user — **so that** streaks, today's dashboard, and reminders all reflect the user's actual local clock.

■ When does it run?

By the reminder worker (to check if it's quiet hours) and by task-service (to filter today's tasks and calculate streaks).

■ Real-life example

Like a world clock app. Your friend in New York asks 'what day is it?' — in Manila it's already Tuesday, but in New York it's still Monday. **timezone.ts makes sure each user's 'today' is calculated in their own local time.**

streak-calculator.ts

src/utils/streak-calculator.ts

■ What is it?

Calculates current streak, best streak, total completions, and completion rate for a task — correctly handling all three frequency types: Daily, Specific Days, and Times Per Week.

■ Why does it exist?

Because a streak is not just 'days in a row'. A habit scheduled for Mondays and Wednesdays only — missing Tuesday doesn't break the streak. **If we used a simple consecutive-day algorithm**, every non-scheduled day would look like a missed day and break the streak incorrectly.

■ Purpose

Calculate meaningful, accurate streak data — **so that** users see honest progress that accounts for their specific schedule.

■ When does it run?

When the stats endpoint is called or the today-dashboard loads.

■ Real-life example

Like a school attendance record. A student has Monday-Wednesday-Friday classes. Missing Tuesday doesn't mean they were absent — Tuesday wasn't a school day. **streak-calculator knows each task's schedule and only counts the days that matter.**

Chapter 4 — Security Layers

Every layer below is active on every request. Think of them as rings of a castle wall.

Layer	What it does	Real-life analogy
Rate Limiting	Max 100 req/15 min global; max 10 on auth routes	Bouncer who lets only N people through per hour
Helmet	Sets 15+ security HTTP headers automatically	A locked display case that prevents tampering
CORS	Only allows requests from your configured frontend	Guest list — unlisted domains turned away
HPP	Blocks HTTP Parameter Pollution attacks	A form clerk who rejects duplicate fields
Body Size Limit	Max 10 KB per request body	Post office refusing oversized packages
Auth Middleware	Validates JWT on every protected route	Concert gate scanner
Zod Validation	Validates every input before it reaches services	Visa form checker
bcrypt	Passwords hashed with cost factor 12	Safe that turns your PIN into an unreadable code
Token Hashing	Refresh/verify tokens stored as SHA-256 hashes	Storing fingerprint instead of the key
Email Verification	Accounts locked until email confirmed	Activating a SIM — you must confirm the OTP
Refresh Rotation	Old refresh token consumed when new one issued	Season pass that renews itself each use
Token Cleanup	Expired tokens deleted nightly	Janitor clearing expired visitor badges
asyncHandler	All async errors forwarded to error middleware	Safety net under tightrope walker
Required ENV vars	App won't start if JWT_SECRET or DATABASE_URL missing	Plane won't take off without fuel confirmed

Chapter 5 — API Endpoints Reference

All endpoints are prefixed with **/api/v1**. Protected routes (■) require Authorization: Bearer <token>.

Method	Endpoint	Auth	What it does
POST	/auth/register	—	Create new account
POST	/auth/login	—	Login — returns token + refreshToken + expiresIn
POST	/auth/logout	—	Revoke refresh token
POST	/auth/verify-email	—	Confirm email with token from inbox
POST	/auth/resend-verification	—	Re-send verification email
POST	/auth/forgot-password	—	Send password reset token
POST	/auth/reset-password	—	Reset password using token
POST	/auth/refresh-token	—	Get new access token using refresh token
GET	/auth/me	■	Get current user profile
GET	/tasks	■	List tasks (paginated: ?page=1&limit=20)
POST	/tasks	■	Create a new task/habit
GET	/tasks/today	■	Today's dashboard with progress %
GET	/tasks/:id	■	Get one task
PATCH	/tasks/:id	■	Update a task
DELETE	/tasks/:id	■	Soft-delete a task
PATCH	/tasks/:id/archive	■	Archive a task
PATCH	/tasks/:id/unarchive	■	Restore from archive
PUT	/tasks/reorder	■	Reorder tasks by drag position
POST	/tasks/:id/complete	■	Mark task complete for today
DELETE	/tasks/:id/complete	■	Undo today's completion
GET	/tasks/:id/completions	■	List all completions for a task
GET	/tasks/:id/stats	■	Streak & statistics for a task
GET	/users/me	■	Get full profile
PATCH	/users/me	■	Update profile (name, phone, city, country, timezone)
PATCH	/users/me/email	■	Request email change (sends token to new email)
POST	/users/me/email/verify	■	Confirm new email with token
PATCH	/users/me/password	■	Change password (current password required)

Chapter 6 — How a Request Travels

Let's trace what happens when a user marks a task as complete. This shows how all the pieces work together.

1. User action	User taps 'Complete' in the app on their phone.
2. HTTP Request	App sends: POST /api/v1/tasks/abc-123/complete with Bearer token in header.
3. Rate Limiter	Counts this IP's recent requests. Under 100/15min — allowed through.
4. Auth Middleware	Reads the token, verifies the JWT signature, extracts userId. Attaches to request.
5. Schema Middleware	Validates the body (date, count, note, mood). All valid — continues.
6. Controller	completeTask() is called. Reads userId and taskId. Passes to service.
7. Service	completeTaskService() verifies the task belongs to this user. Calls repository.
8. Repository	taskCompletionRepository.recordCompletion() — inserts or updates the DB record.
9. Database	PostgreSQL saves the completion row. Returns the saved data.
10. Response	Result flows back up: Repository → Service → Controller → HTTP Response.
11. User sees	App receives {status: 'success', data: {completion: {...}}} — task marked ■.

Chapter 7 — Key Design Decisions

Why soft-delete instead of hard-delete?

When a user deletes a task, we set deletedAt = now() instead of removing the row. Because if we deleted the row, all past completions (linked to that task) would also disappear. The user would lose their history. Soft-delete preserves the data while hiding the task from the UI.

Why store tokens as SHA-256 hashes?

If the database were ever breached, an attacker would find hashes — not the actual tokens. Because SHA-256 is one-way (you cannot reverse it), the attacker cannot use the hashes to log in. The real token only ever exists in the user's email or memory.

Why two tokens (access + refresh)?

Access tokens are short-lived (10 min) so that if one is stolen, it expires quickly. But asking users to log in every 10 minutes is terrible UX. Refresh tokens (7 days) allow silent re-authentication. If a refresh token is stolen and rotated, the old one becomes invalid — the real user's next request will fail, alerting them to re-login.

Why separate repositories from services?

Services contain 'what should happen'. Repositories contain 'how to ask the database'. If we switch from PostgreSQL to another database, we only rewrite repositories — services stay identical. This separation is called the Repository Pattern.

Why enforce email verification before login?

Without verification, anyone could register with your email and potentially receive account-related communications or lock you out. Enforcing verification ensures only the real inbox owner can activate and use the account.

Why paginate the task list?

What if a power user creates 500 tasks over 5 years? Returning all 500 in one response is slow, expensive, and wasteful. Pagination lets the frontend request only what it needs to display — 20 tasks at a time.

Chapter 8 — Complete File Summary

File	Layer	One-line purpose
server.ts	Entry	Starts the server and reminder worker
app.ts	Entry	Security, middleware, route registration
config/env.ts	Config	Loads and validates environment variables
db/schema.ts	DB	Database table blueprints
db/index.ts	DB	Shared PostgreSQL connection pool
middlewares/auth-middleware	Security	JWT token verification on protected routes
middlewares/schema-middleware	Security	Zod request body validation
middlewares/error-middleware	Reliability	Catches all unhandled errors
middlewares/async-handler	Reliability	Auto-forwards async errors to error handler
schema/auth-schema	Validation	Auth endpoint input rules
schema/task-schema	Validation	Task endpoint input rules
schema/user-settings-schema	Validation	Settings endpoint input rules
services/auth/Sign-Up	Service	New account creation + verification email
services/auth/Log-In	Service	Authenticate + issue tokens
services/auth/Email-Verify	Service	Send/verify email confirmation token
services/auth/Forgot-Password	Service	Send/consume password reset token
services/auth/Refresh-Token	Service	Token refresh + logout
services/task/task-service	Service	All task CRUD, dashboard, stats
services/task/completion-svc	Service	Record/undo/list daily completions
services/reminder/worker	Background	Smart email reminder engine
services/user/settings-svc	Service	Profile, email, password management
repositories/user-repo	DB Access	User account DB queries
repositories/user-settings-repo	DB Access	Profile update DB queries
repositories/token-repo	DB Access	Token DB queries + expired cleanup
repositories/task-repo	DB Access	Task DB queries with pagination
repositories/completion-repo	DB Access	Completion DB queries
utils/mailer	Util	Unified email sending
utils/timezone	Util	User-local date/time helpers
utils/streak-calculator	Util	Current/best streak calculations
routes/auth-routes	Routes	Auth URL + method definitions
routes/task-routes	Routes	Task URL + method definitions
routes/user-routes	Routes	User settings URL + method definitions

End of Documentation

Rootine Backend · Cordova Public College · Capstone Project 2026

Imperial, Kenneth R. · Aying, Clifford S.

This document was generated directly from the source code of the Rootine backend. Every explanation reflects the actual implementation — not a plan, but a description of what was built, why it was built that way, and how it all works together.